10 Streams, persistence queries, and projections

This chapter covers

- Creating a simple stream from its basic components: source, flow, and sink
- Reading events from the journal of the persistence entities
- Writing events from the journal into another table

In this chapter, you learn how to read from the journal and create views from it. To do this, Akka uses Akka Streams explicitly to read the journal—with the persistence-query module—and implicitly to create views—with the projections module.

NOTE The source code for this chapter is available at www.manning.com/books/akka-in-action-second-edition or https://github.com/franciscolopezsancho/akka-topics/tree/main/chapter10a. You can find the contents of any snippet or listing in the .scala file with the same name as the class, object, or trait.

10.1 Akka Streams

A *stream* is a sequence of data. It can be infinite, but it doesn't have to be. Examples include the messages you send or receive via email, tweets posted each day, clicks on a website, and videos you watch. From the perspective of the application that processes all of these events, they sometimes involve a huge amount of interaction.

There are a few problems when interacting with live events or a constant flow of information. Sometimes the flow is infinite by definition, so you can't wait until the process is finished to process its contents. Other flows, like audio, are not infinite, but you don't want to wait until the whole song or video is transferred to play it.

That's why there are streaming libraries that allow you to interact element by element in a potentially unlimited stream. But once you start consuming the stream, you inevitably run into the next problem. If the stream producer is faster than the consumer, two things can happen: the consumer takes each new item and keeps it in memory until it runs out of memory and crashes, or you can set an upper limit above which the consumer will discard messages and ask for them to be resent (like TCP). The first option is obviously a problem, but the second may not be as clear. A fast producer that is constantly asked to resend messages must keep an ever-growing list of messages to resend. In the end, it has the same problem as the slow consumer: if the producer doesn't drop the messages, it will crash.

The Reactive Streams initiative on which Akka Streams is based was created with this dual problem in mind. The main idea is to enable "live" streaming through asynchronous communication and backpressure. You're probably familiar with the asynchronous part: communication is nonblocking, meaning the producer doesn't have to synchronize with the consumer. Sending or requesting messages happens independently.

Its other essential property, backpressure, is an effect achieved by a combination of pulling and pushing. From this perspective, a producer cannot send a message to a consumer without first receiving